

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

LEETCODE PROBLEM SOLVING

NAME : SALVA SOUMYA

REGISTER NUMBER : 23MID0061

COURSE TITLE : ADVANCED PYTHON PROGRAMMING

COURSE CODE : CSI3007

FACULTY NAME : SHARMILA BANU K

LAB SLOT : L31+L32+L49+L50

SUBMISSION DATE : 19-08-2026

LEETCODE QUESTIONS SOLVING USING OBJECT ORIENTED PROGRAMMING

QUESTION : 463. Island Perimeter (463) – EASY LEVEL

LINK : https://leetcode.com/problems/island-perimeter/description/

You are given a rectangular row x col grid representing a map. Each cell in the grid contains either:

- 1 representing land
- 0 representing water

Grid cells are connected horizontally or vertically, but not diagonally. The grid is completely surrounded by water, and there is exactly one island in the grid.The island does not contain any lakes, meaning that any water inside the island is not connected to the water surrounding the island.Each grid cell represents a square with a side length of 1.

Your task is to determine the perimeter of the island.

Create a class named IslandPerimeter.The class should contain:

- A constructor to store the given grid.
- A method calculate_perimeter() to calculate and return the perimeter of the island.

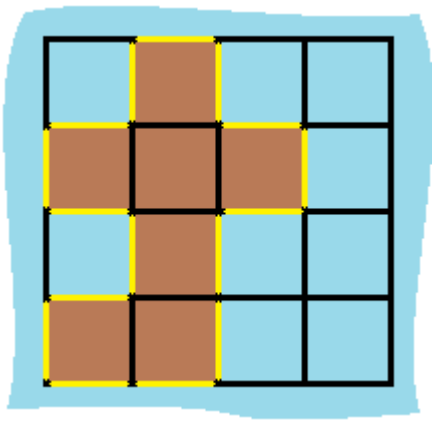
Input format

The input consists of a rectangular 2D grid of integers.

- 1 represents land.
- 0 represents water.

Output format

Return a single integer representing the perimeter of the island**.**



TEST CASES :

Example 1

Input

grid = [[0,1,0,0],
[1,1,1,0],
[0,1,0,0],
[1,1,0,0]]

Output

16

Explanation

The island consists of connected land cells. Each side of a land cell that touches water or the boundary contributes 1 to the perimeter.Therefore, the total perimeter of the island is 16.

Example 2

Input

grid = [[1]]

Output

4

Explanation

There is only one land cell. All four sides are exposed to water.Therefore, the perimeter is 4.

Logic

1. Set perimeter = 0 to store the total perimeter.
2. Traverse every cell of the grid using row and column indices.
3. If the current cell is 0, it represents water, so move to the next cell.
4. If the current cell is 1, it represents land. Check its four sides: up, down, left, and right.
5. If a side is outside the grid or the neighbouring cell is 0, increase perimeter by 1.
6. If the neighbouring cell is 1, do not increase the perimeter because that side is shared by two land cells.
7. Continue this process for all land cells in the grid.
8. **example:**
9. For grid = [[1]], the cell is land and all four sides are exposed. Therefore, the perimeter increases four times and becomes 4.
10. For two adjacent land cells, such as [[1,1]], the common side is not counted. Hence, only the six outer sides contribute to the perimeter.
11. The calculation rule is:

Total Perimeter = Sum of all exposed sides of land cells

11. After checking the complete grid, return the final perimeter value.

QUESTION : 13. Roman To Integer - EASY LEVEL

LINK : https://leetcode.com/problems/roman-to-integer/description/

Create a class named RomanToInteger to represent a Roman numeral and convert it into its corresponding integer value.The class should store the Easy given Roman numeral and provide a method named convert() that performs the conversion and returns the integer value.

Roman numerals are represented by seven different symbols:

Symbol	Value
I	1
V	5
X	10
L	50
C	100
D	500
M	1000

Roman numerals are usually written from largest to smallest value from left to right. However, when a smaller value is placed before a larger value, the smaller value is subtracted.For example:

- IV = 4
- IX = 9
- XL = 40
- XC = 90
- CD = 400
- CM = 900

Implement the convert() method to convert the given valid Roman numeral into its corresponding integer.Create an object of the RomanToInteger class and use the convert() method to obtain the result.

Input Format

The input consists of a string s representing a valid Roman numeral.

Output Format

Return the integer value represented by the given Roman numeral.

TEST CASES :

Example 1

Input

s = "III"

Output

3

Explanation

III = I + I + I = 1 + 1 + 1 = 3

Example 2

Input

s = "LVIII"

Output

58

Explanation

L = 50
V = 5
III = 3

50 + 5 + 3 = 58

Logic

1. Create a mapping for the seven Roman symbols and their corresponding integer values.
2. Initialize result = 0.
3. Traverse the Roman numeral from left to right.
4. Get the integer value of the current symbol.
5. Compare the current value with the value of the next symbol.
6. If the current value is smaller than the next value, subtract it from result.
7. Otherwise, add it to result.
8. Continue until all symbols are processed.
9. For example, for IV, I = 1 and V = 5. Since 1 < 5, subtract 1 and then add 5.
10. Therefore:

    result = 0
    result = 0 - 1
    result = -1 + 5
    result = 4

11. Return the final result from the convert() method.

Calculation Rule

If current value < next value:
  result = result - current value

Otherwise:
  result = result + current value

QUESTION : 13. Container with most water - MEDIUM LEVEL

LINK:https://leetcode.com/problems/container-with-most-water/description/

Create a class named WaterContainer to represent a set of vertical lines and determine the maximum amount of water that can be stored between any two lines.The class should store the given array height and provide a method named max_water() to find and return the maximum amount of water that the container can store.

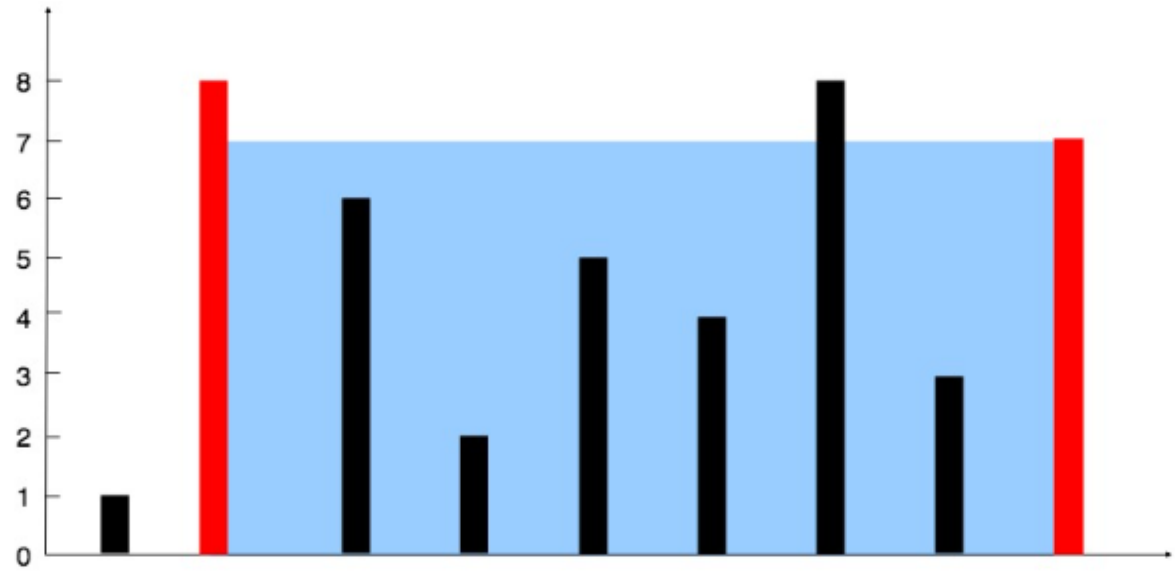
You are given an integer array height of length n. There are n vertical lines such that the two endpoints of the ith line are (i, 0) and (i, height[i]).Choose two lines that, together with the x-axis, form a container that can hold the maximum amount of water.The container cannot be slanted.

Input Format

The input consists of an integer array height containing the heights of the vertical lines.

Output Format

Return the maximum amount of water that the container can store.



TEST CASES :

Example 1

Input

height = [1,8,6,2,5,4,8,3,7]

Output

49

Explanation

The two selected lines have heights 8 and 7.The distance between them is 7.The water level is determined by the shorter line.

Water = min(8, 7) × 7 = 7 × 7 = 49

Example 2

Input

height = [1,1]

Output

1

Explanation

There are two lines with height 1.The distance between them is 1.

Water = min(1,1) × 1= 1

Logic

1. Store the given height array in the class.
2. Use two pointers: left at the first line and right at the last line.
3. Calculate the distance between the two lines:

   width = right - left

4. The water level is determined by the shorter line:

   water_height = min(height[left], height[right])

5. Calculate the water stored between the two lines:

   area = width × water_height

6. If the calculated area is greater than the current maximum, update max_area.
7. Move the pointer pointing to the shorter line towards the other side. This gives a chance to find a taller line and increase the water capacity.
8. Continue until the left and right pointers meet.
9. The maximum value stored in max_area is the maximum amount of water the container can hold.

Formula

Area = min(height[left], height[right]) × (right - left)

QUESTION : 56. Merge Intervals – MEDIUM LEVEL

LINK : https://leetcode.com/problems/merge-intervals/description/

Create a class named IntervalMerger to manage a collection of intervals and merge all overlapping intervals.The class should store the given intervals and provide a method named merge() that returns an array of non-overlapping intervals covering all the intervals in the input.

Each interval is represented as [start, end], where start is the starting point and end is the ending point.Two intervals are considered overlapping when the starting point of the next interval is less than or equal to the ending point of the current interval.Create an object of the IntervalMerger class and use the merge() method to obtain the result.

Input Format

The input consists of a 2D array intervals, where: intervals[i] = [starti, endi]

Output Format

Return an array containing the merged, non-overlapping intervals.

TEST CASES :

Example 1

Input

intervals = [[1,3],[2,6],[8,10],[15,18]]

Output

[[1,6],[8,10],[15,18]]

Explanation

The intervals [1,3] and [2,6] overlap because 2 <= 3.

They are merged into: [1,6]

The other intervals do not overlap, so they remain unchanged.

Example 2

Input

intervals = [[1,4],[4,5]]

Output

[[1,5]]

Explanation

The intervals [1,4] and [4,5] are considered overlapping because the starting point of the second interval is equal to the ending point of the first interval.

Therefore, they are merged into : [1,5]

Logic

1. Store the given intervals in the class.
2. Sort the intervals based on their starting values.
3. Add the first interval to the result list.
4. Take the next interval and compare its starting value with the ending value of the last interval in the result.
5. If the starting value of the current interval is less than or equal to the ending value of the last interval, the two intervals overlap.

6. Merge the overlapping intervals by keeping the smaller starting value and the larger ending value.

   end = max(current_end, next_end)

7. If the intervals do not overlap, add the current interval to the result list.
8. Continue this process until all intervals are checked.
9. Return the result list containing all merged, non-overlapping intervals.

Merging Rule

If:

current_start <= previous_end

then merge:

merged_start = previous_start
merged_end = max(previous_end, current_end)

Otherwise, keep the intervals separate.

QUESTION : 212. Word Seach – HARD LEVEL

LINK : https://leetcode.com/problems/word-search-ii/description/

Create a class named WordSearch to search for a list of words in a given character board.

The class should store the given board and words and provide a method named find_words() that returns all the words that can be formed on the board.Each word must be formed using letters from sequentially adjacent cells. Adjacent cells can be connected horizontally or vertically, but not diagonally.A cell can be used only once while forming a particular word.

Create an object of the WordSearch class and use the find_words() method to obtain all the words present on the board.

Input Format

The input consists of:

- An m × n character matrix board.
- A list of strings words.

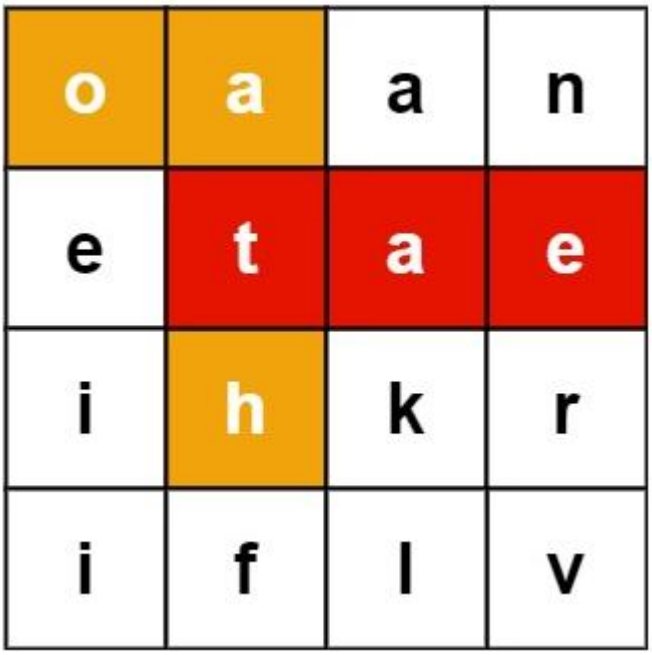
Each cell of the board contains a lowercase English letter.

Output Format

Return a list containing all the words that can be formed on the board.

TEST CASES

Example 1



Input

```
board = [["o","a","a","n"],
        ["e","t","a","e"],
        ["i","h","k","r"],
        ["i","f","l","v"]]

words = ["oath","pea","eat","rain"]
```

Output

["eat","oath"]

Explanation

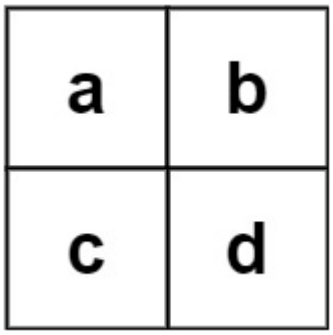
The word "oath" can be formed by following adjacent cells: o → a → t → h

The word "eat" can also be formed using adjacent cells.

The words "pea" and "rain" cannot be formed.

Therefore, the output is: ["eat","oath"]

Example 2



Input

board = [["a","b"],
["c","d"]]

words = ["abcb"]

Output

[ ]

Explanation

Although a → b → c can be followed using adjacent cells, the cell containing b would need to be used again to complete "abcb".

A cell cannot be used more than once for the same word.

Therefore, no word can be formed.

Working Logic

1. Store the board and words in the WordSearch class.
2. Create a data structure to efficiently store and search the given words.
3. Start from every cell of the board.
4. For each cell, try to form a word by moving to its adjacent cells.
5. At each step, check the four possible directions: up, down, left, and right.

6. Continue the search only when the current sequence can form part of a given word.
7. Mark the current cell as visited while searching so that it cannot be used again in the same word.
8. After exploring the cell's neighbours, remove the visited mark so the cell can be used in another search.
9. When a complete word from the given list is formed, add it to the result.
10. Continue searching from all cells until all possible words have been checked.
11. Return the list of words found on the board.

Search Rule

A word can be added to the result only when:

- Its letters are formed in the correct order.
- Consecutive letters are horizontally or vertically adjacent.
- The same cell is not used more than once for that word.

QUESTION – 295. Find Median from Data Stream – HARD LEVEL

LINK : https://leetcode.com/problems/find-median-from-data-stream/description/

Create a class named MedianFinder to maintain a stream of integers and find the median of all the numbers added so far.The class should provide the following methods add_num(num) which adds an integer num to the data stream also find_median() which returns the median of all the numbers currently present in the data stream.

The median is the middle value in an ordered list of integers.If the number of elements is odd, the median is the middle value.If the number of elements is even, the median is the average of the two middle values.

Input Format

The input consists of a sequence of operations performed on the MedianFinder object.

The operations can be:

- add_num(num) — adds a number to the data stream.
- find_median() — finds the current median.

Output Format

For every find_median() operation, return the median of all elements added so far.

TEST CASES

Example 1

Input

Operations:
MedianFinder()
add_num(1)
add_num(2)
find_median()
add_num(3)
find_median()

Output

[null, null, null, 1.5, null, 2.0]

Explanation

After adding 1:

[1]

Median = 1

After adding 2:

[1,2]

Median:

(1 + 2) / 2 = 1.5

After adding 3:

[1,2,3]

Median = 2

Therefore, the median values returned are:

1.5
2.0

Example 2

Input

Operations:

MedianFinder()
add_num(5)
add_num(10)
add_num(15)
find_median()

Output

[null, null, null, null, 10.0]

Explanation

The numbers are:

[5,10,15]

The middle value is 10.

Therefore:

Median = 10.0

Logic

1. Create the MedianFinder class and initialize a data structure to store the numbers.
2. When add_num(num) is called, add the given number to the data structure while maintaining the numbers in sorted order or an appropriate structure for finding the middle efficiently.
3. When find_median() is called, determine the number of elements currently stored.
4. If the number of elements is odd, select the middle element as the median.

   Median = middle element

5. If the number of elements is even, take the two middle elements and calculate their average.

   Median = (middle1 + middle2) / 2

6. Return the calculated median.